

Лабораторная работа 5

Имитационное моделирование

Богданюк Анна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Базовые элементы и принцип работы	7
3.2	Формальное определение	8
3.3	Обедающие философы	8
4	Классическая постановка	9
5	Проблема взаимной блокировки (deadlock)	10
6	Решения проблемы	11
7	Выполнение лабораторной работы	12
8	Выводы	18
9	Список литературы	19

Список иллюстраций

7.1	Создаём рабочее пространство	12
7.2	Создаём src/DiningPhilosophers.jl	12
7.3	Базовый эксперимент	12
7.4	Скрипта для создания анимации и отчёт	13
7.5	Создаём производные форматы	14
7.6	Результат scripts/dining_philosophers.jl	15
7.7	Результат scripts/dining_philosophers.jl	16
7.8	Результат scripts/dining_philosophers_animation.jl	17
7.9	Результат scripts/dining_philosophers_report.jl	17

Список таблиц

1 Цель работы

- Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- Модифицировать сеть, чтобы предотвратить deadlock.
- Проанализировать результаты и оформить отчёт с графиками и анимацией.

2 Задание

1. Создать рабочий каталог для всего курса.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 6. чистый код;
 7. jupyter notebook;
 8. документацию в формате Quarto.
9. Выполнить код из jupyter notebook.
10. Интегрировать документацию в формате Quarto в отчёт.
11. Добавить в код в литературном стиле вычисление для набора параметров.
12. Сгенерировать из литературного кода с параметрами:
 13. чистый код;
 14. jupyter notebook;
 15. документацию в формате Quarto.
16. Выполнить код из jupyter notebook с параметрами.
17. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 Базовые элементы и принцип работы

В основе сети Петри лежит двудольный ориентированный мультиграф — это означает, что вершины в ней делятся на два типа, дуги всегда соединяют вершины разных типов, и между двумя вершинами может быть несколько дуг.

Позиции (Places) — «Условия»

Пассивные элементы, которые обозначаются кружками. Они описывают состояния системы, наличие ресурса или выполнение некоторого условия.

Метки (Tokens): Внутри позиции могут находиться фишки (маркеры), обозначаемые точками или числом. Они символизируют количество ресурсов или кратность выполнения условия. Состояние сети определяется распределением фишек по всем позициям и называется маркировкой.

Переходы (Transitions) — «События»

Активные элементы, обозначаемые прямоугольниками. Они моделируют действия, события или процессы, которые могут произойти в системе.

Дуги (Arcs) — «Связи»

Направленные связи между позициями и переходами. - Входные дуги (от позиции к переходу) указывают, какие условия и сколько фишек необходимо для срабатывания события. - Выходные дуги (от перехода к позиции) показывают, какие условия станут истинными после события и сколько фишек там появится.

Срабатывание перехода есть атомарная операция, которая происходит мгновенно и меняет маркировку сети. Если разрешено несколько переходов, любой из них может сработать.

3.2 Формальное определение

Сеть Петри (N-схема) — это четвёрка: $\Sigma = (P, T, F, M_0)$,
 где: $P = \{p_1, p_2, \dots, p_n\}$ — конечное множество позиций (мест); $T = \{t_1, t_2, \dots, t_m\}$ — конечное множество переходов; $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг (отношения инцидентности); $M_0 : P \rightarrow \mathbb{N}$ — маркировка, задающая начальное распределение фишек

3.3 Обедающие философы

4 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов. Правила поведения философов: - Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест. - Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа. - Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся. - Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

Ограничения: - Вилками нельзя пользоваться одновременно двум философам. - Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит

5 Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть взаимная блокировка.

6 Решения проблемы

Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно.

7 Выполнение лабораторной работы

Создаём рабочее пространство для работы (рис. 7.1).

```
avbogdanyuk@COMPUTER: ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05$ nano setup_project.jl
avbogdanyuk@COMPUTER: ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05$ julia setup_project.jl
```

Рисунок 7.1: Создаём рабочее пространство

Создаём код модели (рис. 7.2).

```
GNU nano 7.2 src/DiningPhilosophers.jl *
add_arc!(net, think, put_forks, +1)
add_arc!(net, left_fork, put_forks, +1)
add_arc!(net, right_fork, put_forks, +1)
end
u0 = zeros(Float64, n_places)
for i = 1:N
    u0[i] = 1.0 # Think_i
    u0[3N+i] = 1.0 # Fork_i
end
return net, u0, net.place_names
end
```

Рисунок 7.2: Создаём src/DiningPhilosophers.jl

Создаём файл scripts/dining_philosophers.jl (рис. 7.3).

```
avbogdanyuk@COMPUTER: ~$ nano scripts/dining_philosophers.jl
avbogdanyuk@COMPUTER: ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers.jl
Activating project at "~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project"
== Классическая сеть (без арбитра) ==
Deadlock обнаружен: true
Call to org.freedesktop.portal.Settings.ReadAll failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
Call for getting org.freedesktop.portal.FileChesser version failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
== Сеть с арбитражем ==
Deadlock обнаружен: false
avbogdanyuk@COMPUTER: ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$
```

Рисунок 7.3: Базовый эксперимент

Зачем?

Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: - классическая модель (без арбитра), в которой возможна взаим-

ная блокировка (deadlock); - модифицированная модель с арбитром, которая должна предотвращать deadlock

Что делает?

- Создаёт классическую сеть Петри для $N=5$ философов с помощью `build_classical_network`.
- Запускает стохастическую симуляцию (алгоритм Гиллеспи) на время $t_{\max} = 50.0$.
- Сохраняет полную историю маркировок в CSV-файл (`dining_classic.csv`).
- Анализирует последнее состояние на предмет deadlock с помощью `detect_deadlock` и выводит результат в консоль.
- Строит графики эволюции маркировки по всем позициям (Think, Hungry, Eat, Fork) и сохраняет их как `classic_simulation.png`.
- Повторяет шаги 1–5 для сети с арбитром (`build_arbiter_network`), сохраняя результаты в `dining_arbiter.csv` и `arbiter_simulation.png`.

Создаём файлы `scripts/dining_philosophers_animation.jl` и `scripts/dining_philosophers_report.jl`. Один для создания анимации эксперимента, а второй - для создания отчёта по результатам базовой эксперимента (рис. 7.4).

```
avbogdan@COMPUTER:~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers_animation.jl
avbogdan@COMPUTER:~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers_animation.jl
Activating project at "~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project"
Call to org.freedesktop.portal.Settings.ReadAll failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
Call for getting org.freedesktop.portal.FileChooser version failed QDBusError("org.freedesktop.DBus.Error.ServiceUnknown", "The name org.freedesktop.portal.Desktop was not provided by any .service files")
[ Info: Saved animation to /home/avbogdan/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
avbogdan@COMPUTER:~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers_report.jl
avbogdan@COMPUTER:~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia scripts/dining_philosophers_report.jl
Activating project at "~/work/study/2026-1/2026-1=study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project"
```

Рисунок 7.4: Скрипта для создания анимации и отчёт

Создаём файлы `scripts/tangel.jl` для того, чтобы преобразовать скрипты в производные форматы и преобразовываем (рис. 7.5).

```

/bogdanyuk@COMPUTER:~/work/study/2026-1/2026-1==study--simulation-mode
/tangle.jl scripts/dining_philosophers_animation.jl
генерация из: scripts/dining_philosophers_animation.jl
Info: generating plain script file from '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.jl'
/ Чистый скрипт: /home/avbogdanyuk/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.jl
Info: generating markdown page from '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.qmd'
/ Quarto: /home/avbogdanyuk/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.qmd
Info: generating notebook from '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.jl'
Info: writing result to '~/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.ipynb'
/ Notebook: /home/avbogdanyuk/work/study/2026-1/2026-1==study--simulation-mode/scripts/dining_philosophers_animation.ipynb

```

Рисунок 7.5: Создаём производные форматы

Результат программы `scripts/dining_philosophers.jl` (рис. 7.6).

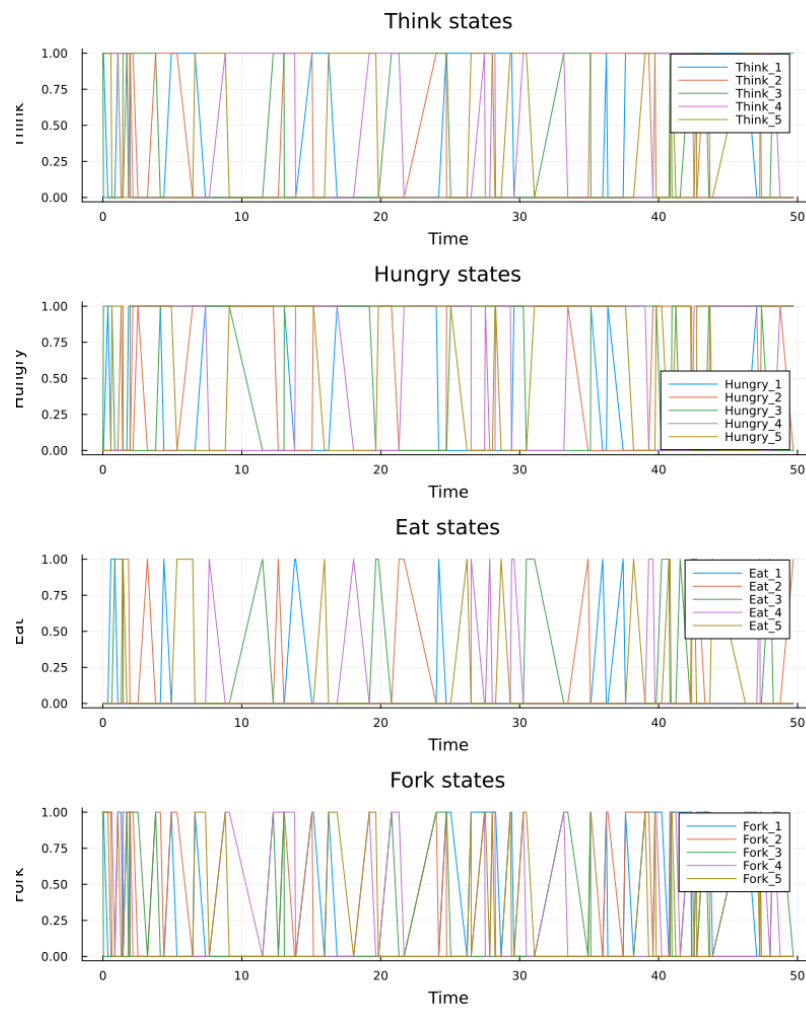


Рисунок 7.6: Результат scripts/dining_philosophers.jl

Результат программы scripts/dining_philosophers.jl (рис. 7.7).

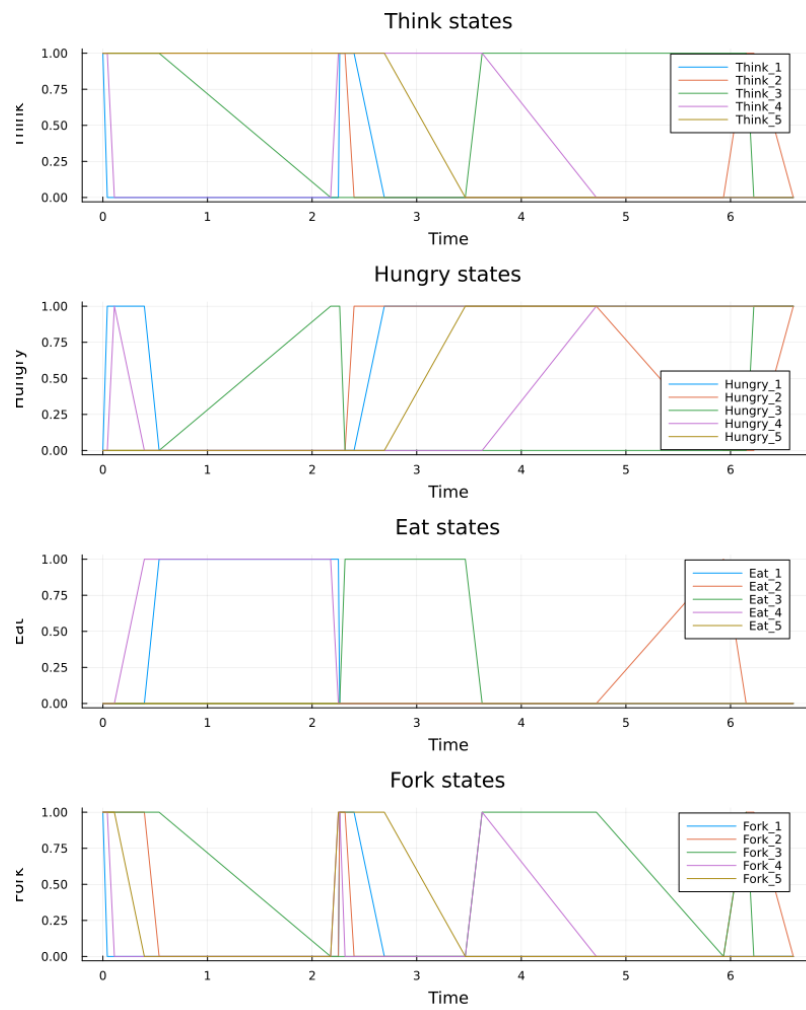


Рисунок 7.7: Результат scripts/dining_philosophers.jl

Результат программы scripts/dining_philosophers_animation.jl (рис. 7.8).

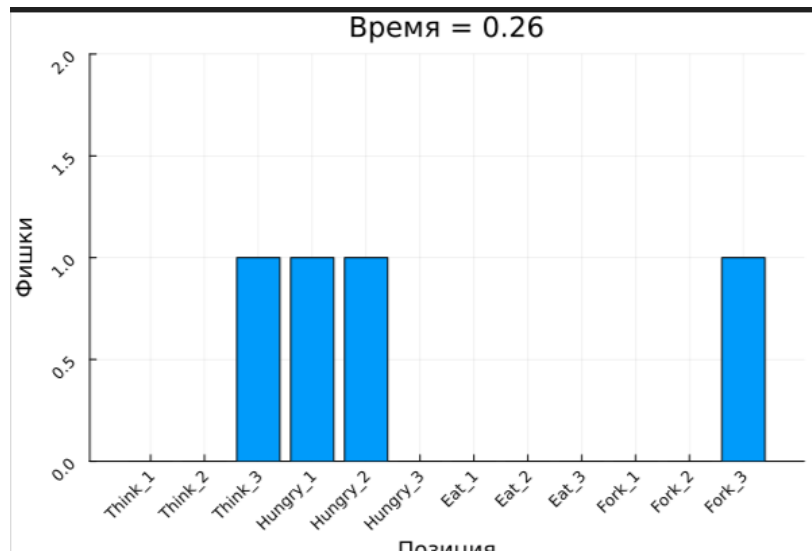


Рисунок 7.8: Результат scripts/dining_philosophers_animation.jl

Результат программы scripts/dining_philosophers_report.jl (рис. 7.9).

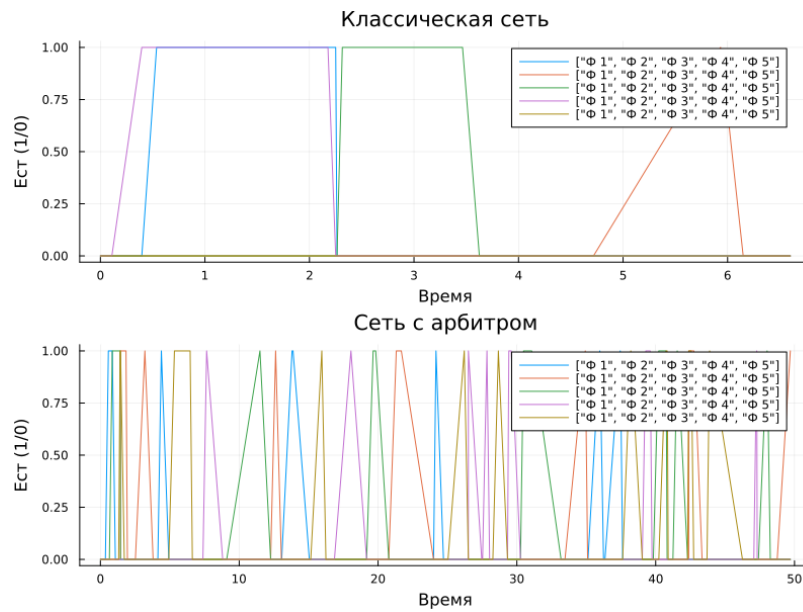


Рисунок 7.9: Результат scripts/dining_philosophers_report.jl

8 Выводы

В ходе выполнения лабораторной работы была построена сеть Петри для пяти философов, моделируя захват и освобождение вилок, обнаружена состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую, было проведено имитационное моделирование (стохастическое и детерминированное) и выявлено наличие deadlock, была модифицировать сеть, чтобы предотвратить deadlock, были проанализированы результаты и был оформлен отчёт с графиками и анимацией.

9 Список литературы

1. Knuth D. E. Literate Programming // The Computer Journal. — 1984. — Feb. — Vol. 27, no. 2. — P. 97–111. — ISSN 1460-2067. — DOI: 10.1093/comjnl/27.2.97.
2. A Multi-Language Computing Environment for Literate Programming and Reproducible Research / E. Schulte [et al.] // Journal of Statistical Software. — 2012. — Vol. 46, no. 3. — ISSN 1548-7660. — DOI: 10.18637/jss.v046.i03.
3. The Story in the Notebook / M. B. Kery [et al.] // Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. — ACM, 04/2018. — P. 1– 11. — DOI: 10.1145/3173574.3173748.